

Computational Geometry: Homework 5

Bishoy Pramanik

Problem 2

Initialization

We can establish **an event queue, a priority queue**, that contains the x-coordinates of all segment endpoints so that they are sorted by x-coordinate. We will maintain a **Status BST** that tracks all segments currently intersecting the sweep line, ordered by their y-coordinate. We also need a bounding box to handle the infinite vertical lines that will come off the segments on the convex hull.

We will add the upper and lower lines of the bounding box to the BST.

We will track the trapezoidal map by keeping a DCEL. We can initialize this by first using the segments and bounding box already provided to us.

For a point **p** in the event queue, we will handle two cases: left and right endpoints.

If p is a left endpoint, we will locate the segment that lies directly above and below p using the BST. On the DCEL, we will create a vertical edge between p and the segment directly above it, as well as a vertical edge between p and the segment directly below it. Finally, we insert the segment that p belongs to into the BST. This splits the current trapezoid to the left of p.

If p is a right endpoint, we will immediately remove the segment that p belongs to from the BST. We then will locate the segments that lie directly above and below p and draw the vertical edges between p and the segments directly above and below it on the DCEL. This closes the trapezoids to the left of p.

Once the event queue is empty, we can return the DCEL that contains the trapezoidal map.

Time Complexity

Constructing the event queue takes $O(n \log n)$ time.

When processing each event, we must perform $\log(n)$ operations on our BST to search, insert, and remove segments on the BST. This gives $O(n \log n)$ time in total for processing all events.

In total, this algorithm is $O(n \log n)$.

Problem 3

a)

In a y-monotone polygon, there must be a distinct top and bottom vertex. We are given the top vertex and all points are sorted in CCW order. We can find the bottom vertex in $\log(n)$ time using the find peak algorithm.

Because the polygon is y-monotone, the distribution of points will be such that the y-coordinates will decrease (we are starting from the top-vertex) until we hit the global minimum (a.k.a. the bottom vertex), and the y-coordinates will increase again. As such we can perform a binary search as follows.

- Check current middle index, q .
 - o If both neighbors of q have a greater y-coordinate than q , then we have found the bottom vertex.
 - o If the left neighbor of q is greater than q , but the right neighbor is less than q , search the right side of q and 'discard' the left side from the search boundary.
 - o If the right neighbor of q is greater than q , but the left neighbor is less than q , search the left side of q and 'discard' the right side from the search boundary.

Because this is a $\log(n)$ search, we can determine the top and bottom points in $\log(n)$ time.

Search Initialization:

We can split the y-monotone polygon into two chains that start at the top vertex and end at the bottom vertex, the left and right chain.

We will initially assess the query point to see if it is to the left or right of the top vertex. If it is to the left of the top vertex, we will search on the left chain, otherwise we will search on the right chain. If it is the same x-coordinate, then we simply need to check if the query point lies below the top vertex, if so, then the query is inside the polygon, otherwise it is not.

Searching:

On a chain, we will now attempt to find the nearest edge to the query point. To do so, we will perform a binary search on the chain. We will search until the search boundary corresponds to an edge, meaning the upper and lower bounds of the search boundary are adjacent points on the chain, forming an edge. Once we have found the nearest edge, we can do a simple check. If we are in a left chain and the query point is to the right of the chain, then the query point is inside the polygon. If we are in a right chain and the query point is to the left of the chain, then the query point is inside the polygon.

Time Complexity

The initial search for the bottom point can be done in $\log(n)$ time. The search initialization is constant time $O(1)$. The search on the chain is $\log(n)$ time, as it is another binary search. In total, this algorithm is $O(\log(n))$.

b)

Once again, we must find the edge that is closest to the query point q . To approach this, we will be using polar coordinates, relative to the witness point, w . Using the first point on our boundary as $\theta = 0$.

First we must calculate the polar coordinates of the query point relative to the witness point on this axis, giving us (r_q, θ_q) . Now we can effectively perform a 1D binary search on the θ -axis along the boundary searching for the edge that best corresponds to the query point, similar to part A. We can search until the search boundary describes two adjacent points that 'contain' the query point. If the query lies inside the edge, then it is inside the polygon, otherwise it is not.

The initialization of polar coordinates is done in constant time $O(1)$.

The binary search along the boundary using polar coordinates is $O(\log(n))$.

In total, this algorithm is $O(\log(n))$.

Problem 4

a) Algorithm Description

We can query the range tree to identify all segments that lie within the vertical line segment, using the top and bottom y-coordinates. This returns all canonical nodes.

For each node, we can query the associated interval tree to ensure that they contain the query x-coordinate. All returned segments that are valid, can be added into a set that stores all valid segments.

Once all canonical nodes have been processed, we can return the set of valid segments.

b) Correctness

The 1D range tree will automatically track and return all segments that are within the specified y-bounds for the search because all valid segments must be within the range $[y_{\text{bottom}}, y_{\text{top}}]$ of the vertical query segment.

Because each canonical subset is returned, we are assessing all possible segments. When we perform out interval query on the canonical subset, we are removing any false positives, ensuring that only the segments that contain the x-coordinate on the query **and** are within the range $[y_{\text{bottom}}, y_{\text{top}}]$ are included. If these two conditions are met, then the segment must be valid and we are returning all valid segments.

c) Time and Space Complexity of Preprocessing, Storage, and Query

Space

A 1D range tree has n points, but each node on the tree contains an associated interval tree. A range tree has a depth of $O(\log(n))$, so each segment appears in $O(\log(n))$ interval trees since it is a descendant of each node on the path to its corresponding leaf node. The path to the leaf is $O(\log(n))$. Not including the interval trees, the space is $O(n)$, but because there is a tree associated at each level, the total space is $O(n \log n)$. Since our equation would resolve to $\sum_{i=1}^{\log n} O(n) = O(n \log n)$.

Preprocessing

Building a 1D range tree takes $O(n \log n)$ time. At each node, building the associated interval tree can take up to $O(n \log n)$ time per level because a range tree contains every segment exactly once per level, giving n nodes per level. There are $\log(n)$ levels. So constructing $\log(n)$ levels where each level requires $O(n \log n)$ work gives $O(n \log^2 n)$ time.

The initial range tree takes $O(n \log n)$ time. The construction of interval trees takes $O(n \log^2 n)$ time. In total, preprocessing takes $O(n \log^2 n)$ time.

Query

The range tree will identify $\log(n)$ canonical nodes for the range bounds. For each node, we query an interval tree, where queries take $O(\log(m) + k)$ time, where $\mathbf{m}$ is the number of segments at that node and $\mathbf{k}$ is the number of reported segments. In total, we get a similar equation:

$$\sum_{i=1}^{\log n} [O(\log n) + k] = \mathbf{O}(\mathbf{log}^2 \mathbf{n} + \mathbf{k}).$$

Problem 6

a) All segments hit

Construction

We can create a segment tree based on the x-coordinates of the segment end points. At each node we can sort the segments vertically since the segments are disjoint to each other. Importantly, this means for a segment s , in the interval spanned by s , it will always be the same relative to the other segments around it. No segment can intersect the segment s , or any other, so all segments maintain their vertical order in their spanned x-coordinate intervals. Each node will contain a sorted list of segments based on this relative vertical order. We can apply fractional cascading to the sorted lists across the tree levels to improve our query time.

Each of the n segments can appear in each of its descendant nodes, giving each n th segment $2\log(2n)$ possible nodes to exist in. In total, we get $n * 2\log(2n)$, or $O(n \log n)$ space complexity.

Query

The initial traversal from the root to the leaf takes $O(\log(n))$ steps. The first binary search at the root also takes $O(\log(n))$ time. Because of the fractional cascading, locating the starting segment in all other nodes along the path takes $O(1)$ per node, totaling $O(\log(n))$. The total time needed to report segments is output sensitive, but we must check the entry point in each of the $O(\log(n))$ nodes even if that specific node returns zero hits. This ensures we can't miss segments stored at different levels of the tree. When we add the k reported answers, the final query time is $O(\log n + k)$.

b) First segment hit

We can create a trapezoidal map to use $O(n)$ storage. Using the properties of a trapezoidal map, we can find the first ray hit by assessing which region the query point is in $\log(n)$ time using the associated DAG. Once the region is identified, we can simply take the top edge from associated face in the DCEL. If the edge hits the boundary, then there is no segment hit.